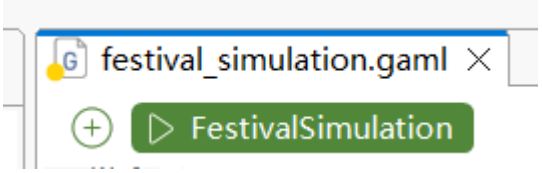
# ID2209 Assignment 1: Festival

Group 3
Shunkang Jia, Rui Zhang, Yuning Liu
shunkang, rzha, yuningl @kth.se

## Section 1: Running Instruction

Copy the festival_simulation.gaml file into user model section and click the "FestivalSimulation" button.




## Section 2: General Overview

In the festival simulation, we designed an information center, guests that may be hungry or thirsty and 2 kinds of stores, serving water or food to the guests. They are all agents.

We have also implemented the 2 challenges: memory of agents & removing bad behavior agents.

In the memory challenge, we enable some guests to "remember" the place it has been to. The agents will then randomly choose between either going to the information center or going directly to the places they remember. We then compared the total distance between guests with or without memory.

In the challenge of removing bad behavior agents, we introduced a new agent, the Security Guard. We randomly generate bad behavior agents. When a guest detects a bad behavior guest, it reports to the information center. Then, the information center calls the Security Guard remotely and tells it who the bad agent is. When the safety guard reached the bad behavior guest, he killed him.

To improve clarity of demonstration, we notate agents with different color, shape and size. We have also introduced the line chart to compare the total distance of 2 types of agents.

## Section 3: Festival Base Model

### 3.1 Explanation

The base model simulates a festival where Guests wander, their hunger/thirst decrease over time, and once either falls below a threshold, the guest goes to the Information Center to ask

for the nearest store of the required type. The guest then travels to that store, replenishes the corresponding need to 100, and returns to an idle state.

Our approach:

· Environment & entities: A square world ( world_size = 100). One InformationCenter at the center; Stores randomly placed with equal chance to be FOOD or WATER; Guests randomly placed.

· Guest behavior:

  · idle -> needs decrease + gentle wandering;

  · if hunger < hunger_threshold or thirst < thirst_threshold -> seeking_info;

  · at information center -> query nearest needed Store and go going_to_store;

  · at store -> replenish ( hunger or thirst = 100 ) -> back to idle.

· Assumptions: World and store placement are random; initial needs are random in [50,100], so guests will naturally trigger the cycle.

## 3.2 Code

### 3.2.1 Global setup & initialization

It creates one Information Center at the center of the world, randomly places multiple Stores with a type of either FOOD or WATER, and generates several Guests with random initial hunger and thirst levels.

```
global {
    // 全局参数
    int nb_guests <- 15;  // 游客数量
    int nb_stores <- 6;   // 商店数量
    float world_size <- 100.0;

    // 信息中心位置（世界中心）
    point info_center_location <- {world_size/2, world_size/2};

    init {
        // 创建信息中心
        create InformationCenter number: 1 {
            location <- info_center_location;
        }

        // 创建商店 - 3个食物店，3个饮料店
        create Store number: nb_stores {
            location <- {rnd(world_size), rnd(world_size)};
            store_type <- flip(0.5) ? "FOOD" : "WATER";
        }

        // 创建游客 - 50%有记忆系统，50%无记忆系统
        create Guest number: nb_guests {
            location <- {rnd(world_size), rnd(world_size)};
            hunger <- rnd(50.0, 100.0);
            thirst <- rnd(50.0, 100.0);
```

### 3.2.2 Guest Behaviour

(1) Guest attributes & state machine

This part defines the guest's basic attributes: hunger, thirst, thresholds, movement speed, and the four possible states. These fields allow the guest to switch between idle, asking for help, going to a store, and replenishing.

```
// 游客代理
species Guest skills: [moving] {
    // 属性
    float hunger <- 100.0 min: 0.0 max: 100.0;
    float thirst <- 100.0 min: 0.0 max: 100.0;
    float hunger_threshold <- 30.0;
    float thirst_threshold <- 30.0;
    This definition of speed supersedes the one in skill moving
    float speed <- 1.0;

    // 状态
    string state <- "idle" among: ["idle", "seeking_info", "going_to_store", "at_store"];
    point target_location <- nil;
    Store target_store <- nil;
```

(2) idle behaviour

While idle, guests slowly wander around and their hunger and thirst decrease. When either need drops below its threshold, the guest switches to seeking_info.

```
reflex decrease_needs when: state = "idle" {
    // 随机减少饥饿和口渴值
    hunger <- hunger - rnd(0.5, 1.5);
    thirst <- thirst - rnd(0.5, 1.5);

    // 随机移动
    do wander speed: speed * 0.3;
}

reflex check_needs when: state = "idle" {
    if (hunger < hunger_threshold or thirst < thirst_threshold) {
        // 挑战1关键改进：有记忆的游客可以直接去商店！
        if (has_memory and length(memory) > 0) {
            // 确定需要什么类型
            string needed_type <- hunger < thirst ? "FOOD" : "WATER";

            // 70%概率使用记忆，30%去信息中心探索新地方
            if (flip(0.7)) {
                // 从记忆中选择合适类型的商店
                list<Store> suitable_memory <- memory where (each.store_type = needed_type);
                if (length(suitable_memory) > 0) {
                    // 直接去记忆中最近的商店！
                    target_store <- suitable_memory closest_to location;
                    target_location <- target_store.location;
                    state <- "going_to_store";
                } else {
                    // 记忆中没有合适的，去信息中心询问
                    state <- "seeking_info";
                }
            } else {
                // 30%概率探索新地方，去信息中心
                state <- "seeking_info";
            }
        } else {
            // 无记忆的游客必须去信息中心
            state <- "seeking_info";
        }
    }
}
```

(3) Visiting the Information Center

When needs become critical, the guest moves toward the Information Center. Once close enough, it uses ask to request the nearest store matching its need (FOOD or WATER).

```
reflex go_to_info_center when: state = "seeking_info" {
    // 前往信息中心
    target_location <- info_center_location;
    do goto target: target_location speed: speed;

    if (location distance_to info_center_location < 2.0) {
        // 到达信息中心，询问最近的商店
        ask InformationCenter at_distance 5.0 {
            // 确定需要什么类型的商店
            string needed_type <- myself.hunger < myself.thirst ? "FOOD" : "WATER";

            // 无论有无记忆，来到这里的都需要询问
            myself.target_store <- self.find_nearest_store(myself.location, needed_type);

            if (myself.target_store != nil) {
                myself.target_location <- myself.target_store.location;
                myself.state <- "going_to_store";
            }
        }
    }
}
```

(4) Going to store & replenishing

The guest travels to the selected store.

Upon arrival, it replenishes hunger or thirst to 100, resets the target, and returns to idle state to continue the cycle.

```
reflex go_to_store when: state = "going_to_store" and target_store != nil {
    do goto target: target_location speed: speed;

    if (location distance_to target_location < 2.0) {
        state <- "at_store";
    }
}

reflex replenish when: state = "at_store" and target_store != nil {
    // 补充需求
    if (target_store.store_type = "FOOD") {
        hunger <- 100.0;
    } else if (target_store.store_type = "WATER") {
        thirst <- 100.0;
    }

    // 挑战1: 只有有记忆的游客才会记住商店
    if (has_memory and !(memory contains target_store)) {
        add target_store to: memory;
    }

    // 重置状态
    state <- "idle";
    target_store <- nil;
    target_location <- nil;
}


aspect default {
    if (is_bad and !reported) {
        draw circle(1.5) color: #red;  // 坏游客 - 红色
    } else if (is_bad and reported) {
        draw circle(1.5) color: #darkred;  // 被举报的坏游客 - 深红色
    } else if (has_memory) {
        draw circle(1) color: #blue;  // 有记忆的游客 - 蓝色
    } else {
        draw circle(1) color: #gray;  // 无记忆的游客 - 灰色
    }

    // 显示状态
    draw string(state) size: 3 color: #black at: location + {0, 2};
}
```

## 3.3 Demonstration

### 3.3.1 Use case 1 —- Hungry Guest Going to Nearest FOOD Store

(1) Input Description

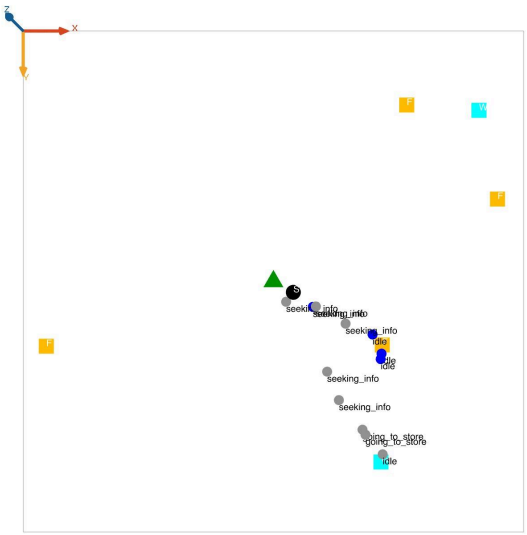
· Run the simulation using default parameters:

· Number of Guests = 15

· Number of Stores = 6

· Wait until one guest's hunger value drops below the threshold, triggering the transition to the seeking state.

(2) Screenshot




(3) Interpretation

In this scenario, one of the guests becomes hungry and transitions from the `idle` state to the `seeking_info` state. As shown in the screenshot, the guest moves toward the green Information Center to request directions. Several guests are currently labeled "seeking_info", which indicates that their hunger or thirst values have dropped below the threshold. Upon reaching the Information Center, the hungry guest will use the `ask` interaction to obtain the location of the nearest FOOD store (orange "F") and then proceed toward it. This demonstrates the correct base-model behavior: guests detect low needs, communicate with the Information Center, and begin traveling to the appropriate store for replenishment.

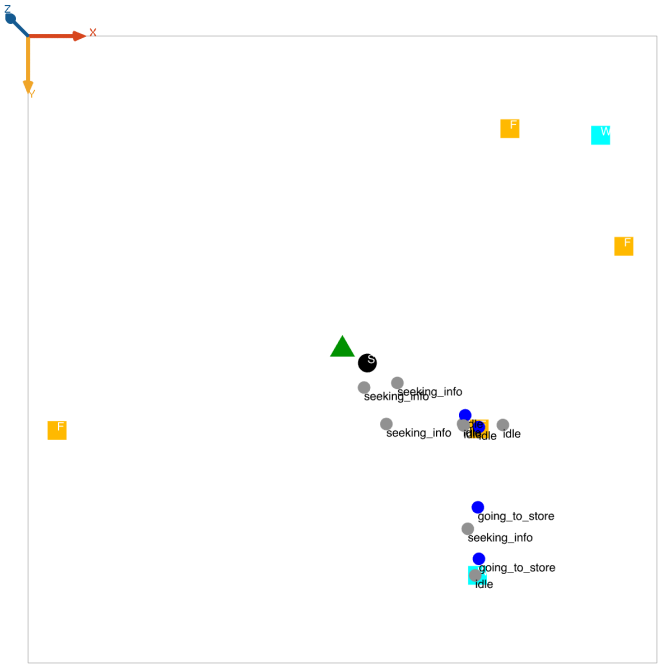
### 3.3.2 Use case 2 —- Thirsty Guest Visiting the Nearest WATER Store

(1) Input Description

· Use the same default simulation settings.

· Wait until a guest's thirst value drops below the threshold.

(2) Screenshot




(3) Interpretation

In this scenario, a guest becomes thirsty and transitions from the `idle` state into `seeking_info`. After reaching the Information Center, the guest receives the location of the nearest WATER store. As shown in the screenshot, the guest's state changes to "going_to_store", and the agent is moving directly toward the cyan WATER store labeled "W". Upon arrival, the guest will replenish its thirst value to 100. This demonstrates the correct base-model behavior in which a thirsty guest requests assistance from the Information Center and then travels to the appropriate store for replenishment.

# Section 4: Challenge 1 Memory of Agents

**4 complete use case**

## 4.1 Explanation

When a guest with memory visits the store, it will remember the store. The next time it is hungry/thirsty, by 70% chance it will choose a store in his memory and by 30% chance it goes to the information center to ask for a new location.

We add one attribute to the guest species: "hasMemory" to tell apart 2 types of agent.

## 4.2 Code

### 4.2.1 Key Addition in Guest state machine: Check needs

In the state of “check_needs”, the guest need to flip a coin and decide whether to visit the regular store or go to the service center for a new one. If the agents has no memory of any store, he will goes to service center directly.

```
reflex check_needs when: state = "idle" {
    if (hunger < hunger_threshold or thirst < thirst_threshold) {
        // 挑战1关键改进：有记忆的游客可以直接去商店！
        if (has_memory and length(memory) > 0) {
            // 确定需要什么类型
            string needed_type <- hunger < thirst ? "FOOD" : "WATER";

            // 70%概率使用记忆，30%去信息中心探索新地方
            if (flip(0.7)) {
                // 从记忆中选择合适类型的商店
                list<Store> suitable_memory <- memory where (each.store_type = needed_type);
                if (length(suitable_memory) > 0) {
                    // 直接去记忆中最近的商店！
                    target_store <- suitable_memory closest_to location;
                    target_location <- target_store.location;
                    state <- "going_to_store";
                } else {
                    // 记忆中没有合适的，去信息中心询问
                    state <- "seeking_info";
                }
            } else {
                // 30%概率探索新地方，去信息中心
                state <- "seeking_info";
            }
        } else {
            // 无记忆的游客必须去信息中心
            state <- "seeking_info";
        }
    }
}
```

### 4.2.1 Key Addition in Guest state machine: Replenish

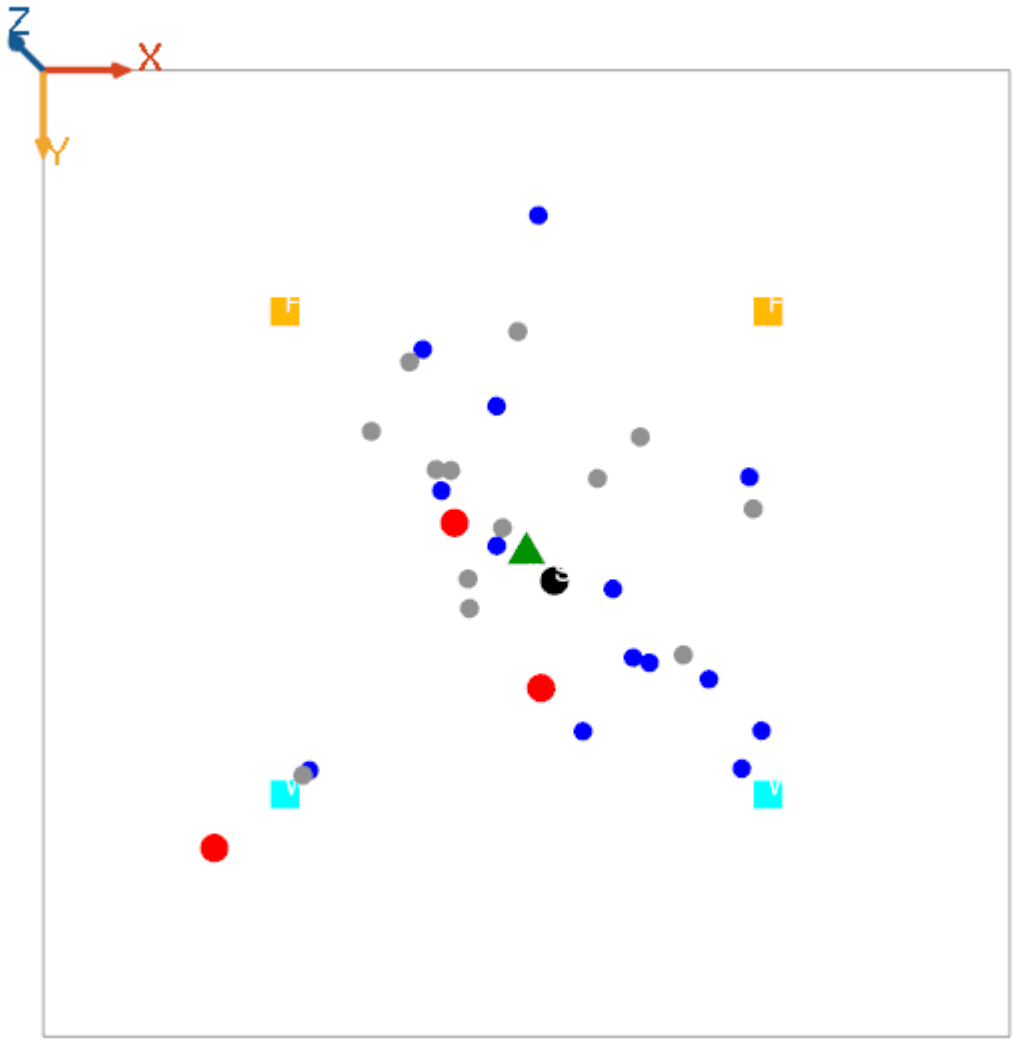
When a guest with memory arrives at the store, he will remember that store.

```
reflex replenish when: state = "at_store" and target_store != nil {
    // 补充需求
    if (target_store.store_type = "FOOD") {
        hunger <- 100.0;
    } else if (target_store.store_type = "WATER") {
        thirst <- 100.0;
    }

    // 挑战1: 只有有记忆的游客才会记住商店
    if (has_memory and !(memory contains target_store)) {
        add target_store to: memory;
    }

    // 重置状态
    state <- "idle";
    target_store <- nil;
    target_location <- nil;
}
```

## 4.3 Demonstration

### 4.3.1 A guest goes to the service center and store it in memory

As is shown in the diagram, every agent, be it blue(with memory) or grey (without memory), has to go to the central green information office for location at the very beginning.

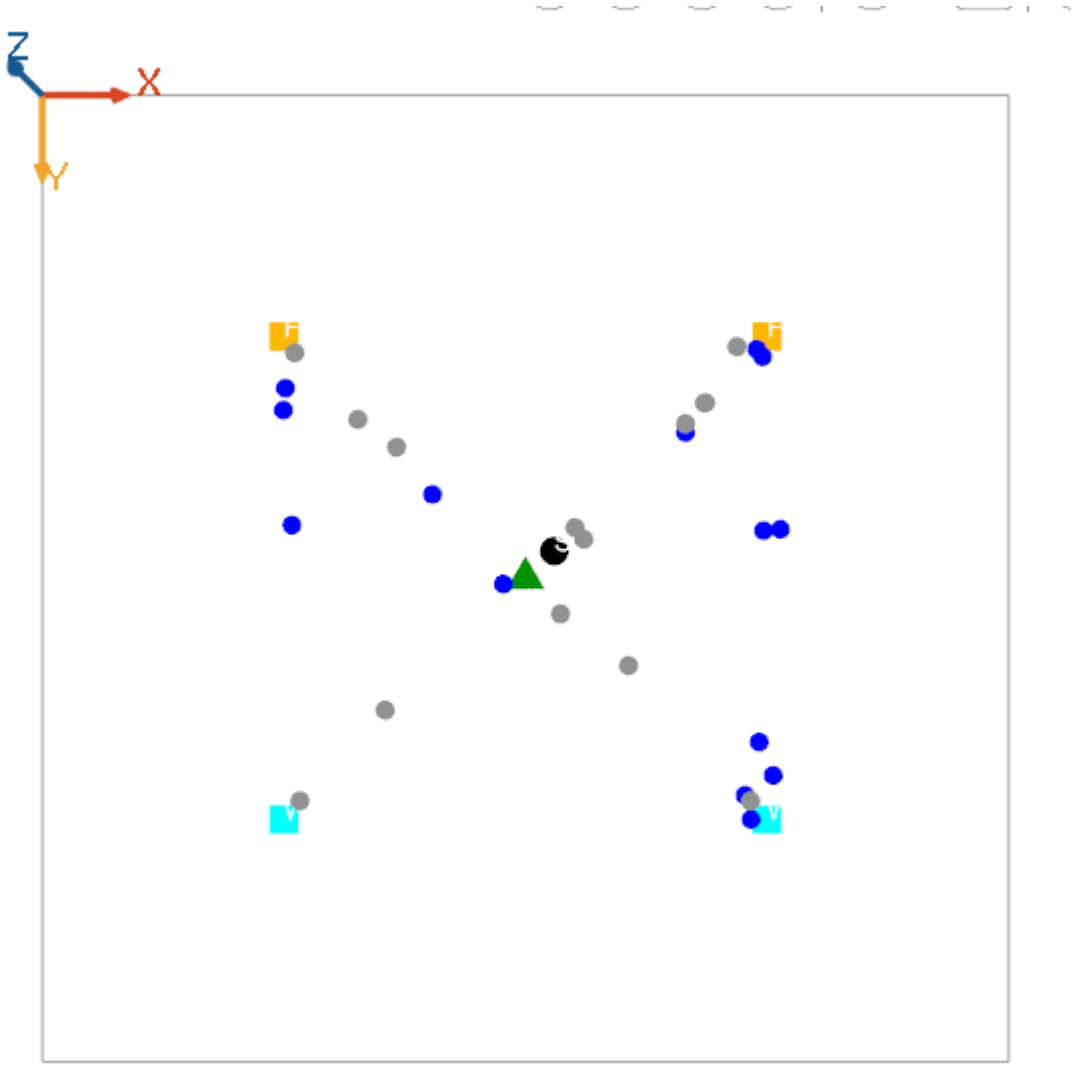



### 4.3.2 A guest flips a coin and decide to go to service center

As is shown in this use case, the blue ones are agent with memory, some of them still go to the green information center for location information.

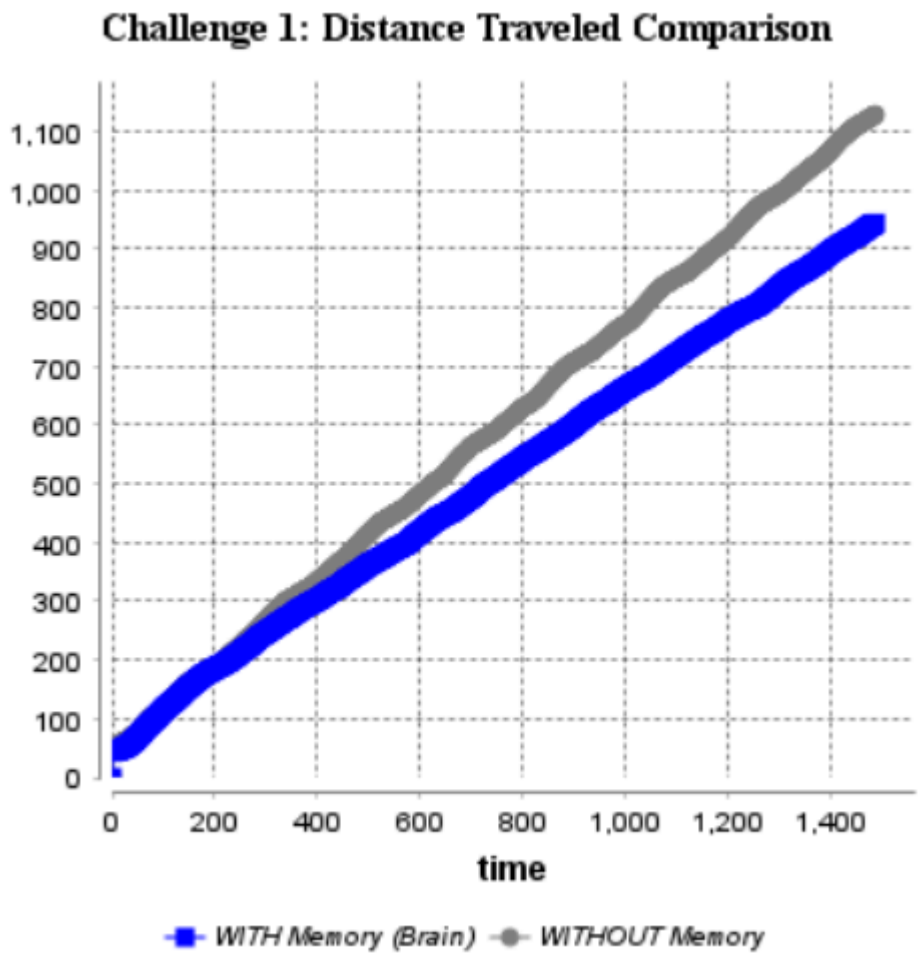
### 4.3.3 A guest flips a coin and decide to go to remembered store

As is shown in this use case, the blue ones are agent with memory, the grey ones are agent without memory. Only the blue one can take the short path from one store to another, while the grey one always have to return to the center green information center to ask for location.




### 4.3.3 Comparison of Total Distance

In our experiment, 50% of guests are with memory and 50% of agents are without memory, so that we can compare the total difference directly. As is shown in the line chart, as time increases, the agents with memory save the distance compared to those without memory.




# Section 5 ：Challenge 2 Removing Bad Behaviour Agents

## 5.1 Explanation

In this challenge, we introduce one new attribute in guest species: isBad. Correspondingly, we notate the bad guest as red. When they are closed to other guests, then in each update, they are 1% to be found and reported by the normal agents to the Information Center.

We also implement a new species Security Guard with faster speed. It has two states: idle and chasing. Every time the information center ask it to chase a bad guest, it turns to chasing state and when the distance to the bad guest within a threshold, he call die function to kill the agent.

## 5.2 Code

### 5.2.1 Security Guard Implementation

```
// 挑战2: 保安代理
species SecurityGuard skills: [moving] {
    Guest target_bad_guest <- nil;
    This definition of speed supersedes the one in skill moving
    float speed <- 2.0;  // 保安移动更快
    string state <- "idle" among: ["idle", "chasing"];

    action chase_bad_guest(Guest bad_guest) {
        target_bad_guest <- bad_guest;
        state <- "chasing";
    }

    reflex chase when: state = "chasing" and target_bad_guest != nil {
        do goto target: target_bad_guest.location speed: speed;

        // 如果接近坏游客，移除他
        if (location distance_to target_bad_guest.location < 2.0) {
            write "Security Guard: Bad guest removed from festival!";
            ask target_bad_guest {
                do die;
            }
            target_bad_guest <- nil;
            state <- "idle";
            // 返回信息中心
            do goto target: info_center_location speed: speed;
        }
    }

    reflex return_to_base when: state = "idle" and location distance_to info_center_location > 5.0 {
        do goto target: info_center_location speed: speed * 0.5;
    }

    aspect default {
        draw circle(1.5) color: #black;  // 保安 - 黑色
        draw "S" size: 4 color: #white at: location;

        if (state = "chasing") {
            draw line([location, target_bad_guest.location]) color: #red width: 2;
        }
    }
}
```

### 5.2.2 Addition to Guest

The guest should be able to detect the bad guests.

```
// 挑战2: 好游客发现坏人后去报告
reflex report_bad_behavior when: !is_bad and flip(0.3) {
    // 在自己周围找还没有被举报的坏人
    list<Guest> bad_nearby <- Guest at_distance 10.0 where(each.is_bad and !each.reported);
    if (length(bad_nearby) > 0) {
        Guest bad_guest <- one_of(bad_nearby);

        // 被报告到信息中心
        ask one_of(InformationCenter) {
            do report_bad_guest(bad_guest);
        }

    }
}
```

### 5.2.3 Addition to Information Center

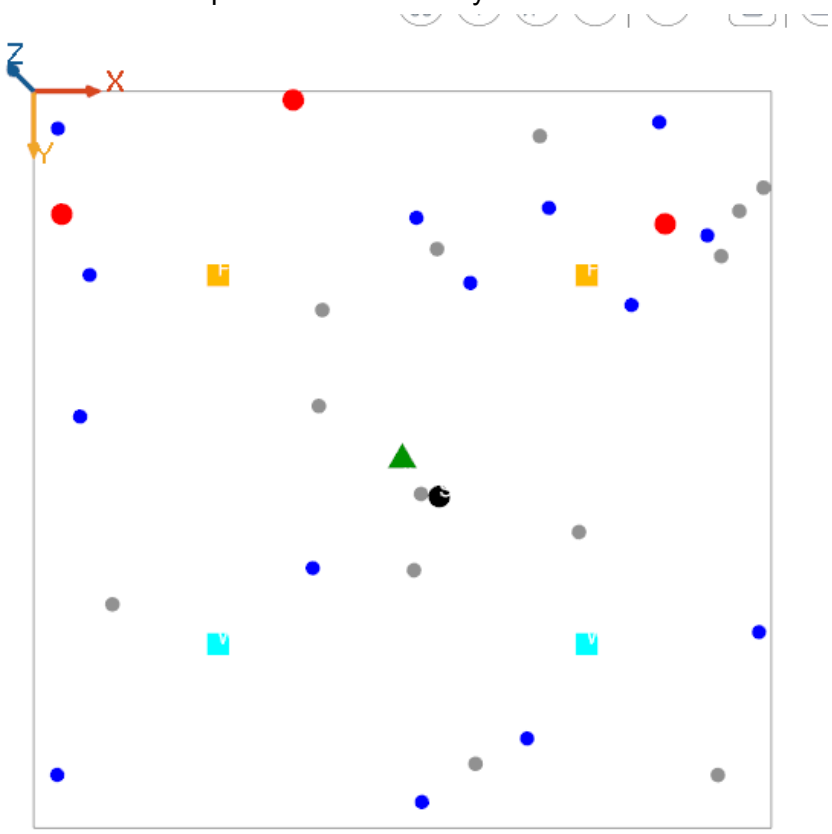
The information center can ask the Security Guard to chase the bad guest.

```
// 挑战2: 报告坏游客
action report_bad_guest(Guest bad_guest) {
    write "Information Center: Bad guest reported! Calling security...";

    // 呼叫保安
    ask SecurityGuard {
        do chase_bad_guest(bad_guest);
    }
}
```

## 5.3 Demonstration

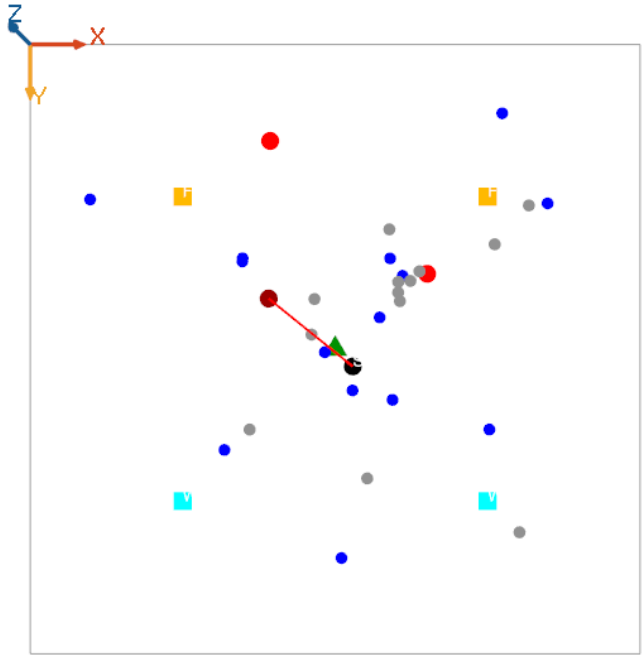
### 5.3.1 Initialize of bad guests and security guard

The red circles represent the bad guest.
The black circle represents the Security Guard.



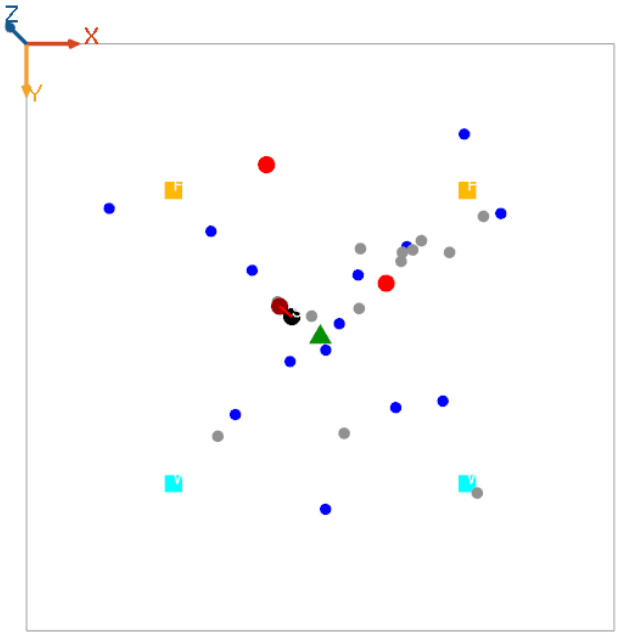

### 5.3.2 Change color when a bad guest is reported

The color of a bad guest is changed to dark red if it is reported.




### 5.3.3 Security Guard start to chase

There is a red line connecting the Security Guard and the reported bad guests to show this is the target.

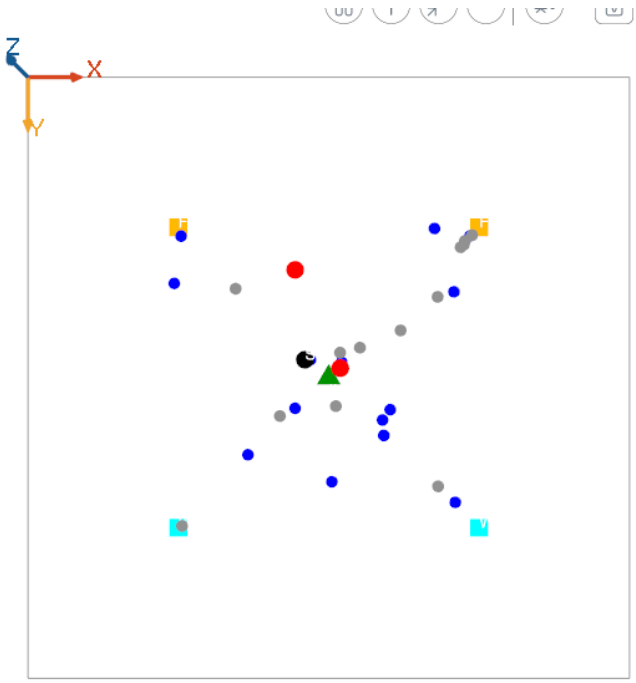


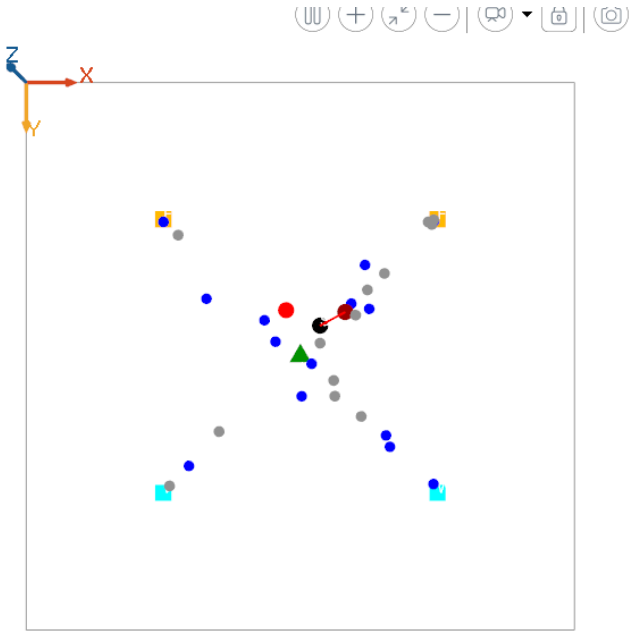

### 5.3.3 Security Guard kills the bad guest

When the guard reached the bad guest(within distance threshold), he kills him.

Console | Interactive console

```
Information Center: Bad guest reported! Calling security...
Security Guard: Bad guest removed from festival!
Information Center: Bad guest reported! Calling security...
Security Guard: Bad guest removed from festival!
```



## Section 6: Conclusion

In this assignment, we have learned how to use the gama platform and programming language. We have also tried out the communication between agents as well as some visualization skills such as plotting charts.

The result has shown that the agent with memory slightly reduces the overall distance.

One of the limitations of work is that we assume that guests always go to the nearest store. However, in the real world it may cause congestion. Maybe a delay can be introduced to the stores when serving guests so that the simulation is closer to the real world.